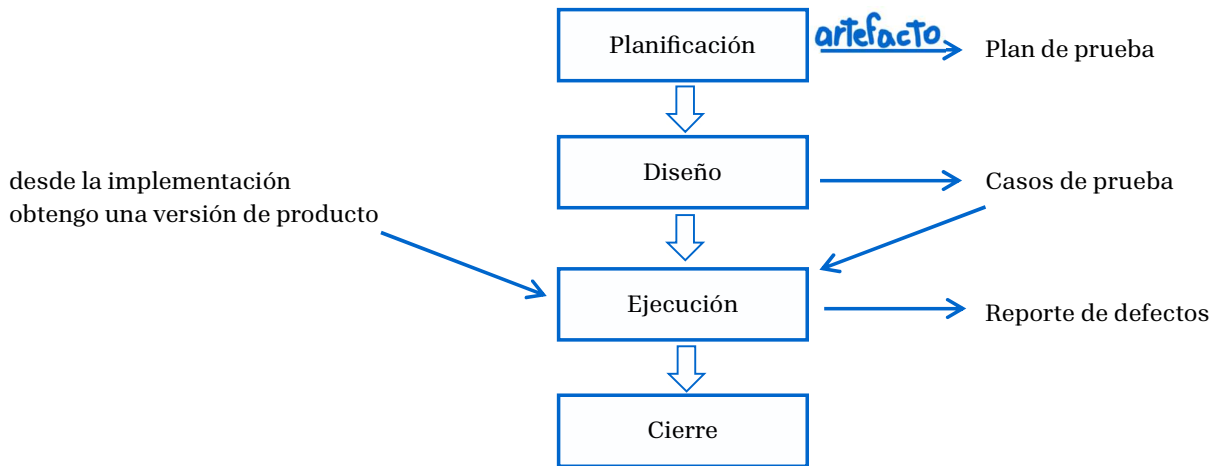


TESTING – PRUEBA

Proceso [PUD]

1. Requerimientos
2. Análisis
3. Diseño
4. Implementación
5. Prueba
6. Despliegue

El Testing está ubicado en la actividad de Prueba del Proceso. Tiene actividades:



El reporte de defectos ingresa a la Implementación para corrección del software.

Los CDP vienen de los Requerimientos, entonces, una vez que los req. están definidos, puedo empezar a planificar estas pruebas definiendo quien lo hará, como lo voy a hacer, cuando, donde, etc. Todas estas preguntas deben estar relacionadas con el testing.

Un testing exitoso es cuando encuentras un defecto y lo reportas al desarrollador

CONCEPTOS IMPORTANTES:

- Testing metodológico VS testing Adhoc
 - Testing metodológico: testing que se hace utilizando un proceso. Diseña y usa los casos de prueba. En el caso de que no tenga CDP:
 - Testing adhoc: se crea cuando se necesita. No tienes forma de reproducirlo, un error que no se puede reproducir no es un error!!! Si no tengo un CDP que me diga paso a paso lo que debo hacer, no lo podré reproducir, por lo que es esfuerzo tirado a la basura. Te sientas y empiezas a testear. Es no metodológico, no tiene proceso.
NO PROBAR SI NO TENGO CDP
- Error VS Defecto

Un error puede generar una falla en el sistema. Lo generamos nosotros, y como consecuencia, hacemos una falla.

- Error: No se traslada de otra etapa, nace en esta etapa y se puede solucionar sin afectar a otra etapa. Son más baratos.
- Defecto: algo es un defecto cuando se trasladó de una etapa anterior a la etapa de la que estamos hablando. Testing recibe un artefacto de la etapa anterior (Implementación). Testing encuentra defectos, porque no se originaron en la etapa de testing.

El testing es un proceso cuyo propósito es la detección de defectos cuya presencia se asume en el código. Siempre hay defectos, porque desarrollamos las personas y es común que cometamos errores.

Hay dos procesos en el testing:

- Validar: es un proceso que tiene que ver con la adecuación. “Vemos si hace lo que realmente tiene que hacer”
- Verificar: es un proceso que comprueba que, si lo hace, lo hace bien. Por ejemplo, si divide, ¿divide bien?

Si hago un testing serio, debo poder llevar a cabo los 2 procesos. Hay en casos que no puedo hacer validación, como con el adhoc. Se da en casos que no se que debe hacer el software, entonces no se si lo está haciendo bien.

Si utilizo los CDP, puedo hacer los dos procesos. Se puede dar que pase ambas, que pase una sola, o que no pase ninguna.

Los procesos de validación son los más caros de corregir. Muchas veces debo arrancar de nuevo. Si el error está en los Requerimientos, debo arrancar todo de nuevo, por lo que es carísimo. Los errores de arquitectura son los 2dos más caros, y tardamos muchísimo en detectarlos.

El testing, se lleva de un proyecto el 30-50% del costo (o al menos eso debería asignársele). La mayoría de las veces, al hacer estimaciones, solo se estima el tiempo de implementación, la cual no debería llevar más del 35% del proyecto. Esto es un problema grave ya que lleva a desviaciones en la planificación.

No se hace suficiente testing porque el testing exhaustivo tampoco es posible hacerlo debido a su costo. "Si como total no puedo hacer testing exhaustivo, mejor no hago nada" y sacan el producto sin probarlo.

Al no hacer testing, el usuario final es quien lo hace cuando lo usa.

La edad del defecto es el tiempo que transcurre desde que se introduce el defecto, hasta que se lo detecto. Mientras más edad tiene el defecto, más caro es resolverlo.

Peer Programming: tiene como objetivo la detección inmediata del defecto. Prestar atención es más agobiante que detectar en sí. Pero esto es más barato. Si llevas a cabo la técnica de forma exitosa, te ahorras en la corrección. 4 ojos ven más que 2 → trabajar de forma colaborativa.

Los defectos tienen categoría, la **Severidad del defecto** no tiene que ver con lo que a mí me gusta corregirlo, tiene que ver con el impacto que provoca eso en el uso del sistema. Que tan grave es el efecto que causa en el uso. Es sistema anda. Estos pueden ser:

- Bloqueante – invalidante: no puedo usar el sistema ya que no anda.
- Grave:
- Leve:
- Clásico cosmético: es atribuido a cuestiones de interacción con la interfaz de usuario o de tipografía. Formateo del CUIT, por ejemplo.

Después hay otros que los usan la gente que hace testing como consejo o mejora. No es un defecto, pero se puede mejorar el sistema.

- Mejora

La prioridad es la urgencia que tiene el cliente. Se puede clasificar en:

- Alta
- Media
- Baja

Siempre ponen alta porque piensan que, de otra forma, no serán resueltos, es por ello que asignan a todo con prioridad alta, pero ahí no están priorizando.

Muchas veces las severidades se negocian con el cliente.

NIVELES DE TESTING

Hay 4 niveles, se prueba de lo más chiquito (nivel 4) a lo más grande (nivel 1), contrario a cuando desarrollamos.

1. Prueba de aceptación de usuario: se hace en el workflow de Despliegue.
2. Sistema: se hace en el workflow de Prueba.
3. Integración: pruebo componentes unitarios que se que funcionan bien individualmente y los integro para ver como funcionan juntos. El resultado es un ¿?? que entra al nivel de sistema. Lo hace el workflow de Prueba, pero muchas veces lo hace el de Implementación.
4. Prueba unitaria: el TDD integra la prueba unitaria, sino seria carísimo. Cuando programo, debo ver si lo que programé anda o no. Lo hace el workflow de Implementación.

Los ambientes de Software son:

- Ambiente de desarrollo: donde trabajan los desarrolladores. Se realizan las pruebas unitarias y de integración
- Ambiente de prueba: deben ser limpios, dedicados solamente a los que prueban y los desarrolladores no deberían tener acceso. Se realizan las pruebas de sistema
- Ambiente de Producción

En la situación ideal, también hay un ambiente que se llama:

- Ambiente de Pre_Producción: donde se hacen las Pruebas de aceptación del Usuario con una cuestión mínima de riesgos. La mayoría de las veces no se hace por una cuestión de costos. Muchas empresas prefieren hacerlas en el ambiente de Producción.

Hay veces que no hay ambiente de producción porque hacemos software para otros (como en el CIDS) ya que no se ejecuta el sistema ahí. Pero si soy un banco, donde tienes un espacio para desarrollar tu propio software, de mínima, debería hacer un ambiente de producción para ejecutar su sistema.

CICLO DE PRUEBA

Es cuando tenemos los Casos de Prueba en Ejecución, iniciando en el ambiente de prueba, corro todos los casos, obtengo mi reporte de defectos y termino mi ciclo, mandándole mi reporte

El 1er ciclo que se ejecuta se llama Ciclo Cero.

La situación ideal en un ambiente de desarrollo debería tener 2 ciclos de prueba, uno que es el ciclo cero que debe estar siempre, para parametrizar la herramienta y encontrar todo lo máximo que debería encontrar. Y otro para solucionar los defectos.

Un ciclo de prueba por cada versión que entra a testing.

Muchas veces al hacer testing se tiene en cuenta 1 solo ciclo, lo cual está mal

Estrategias de prueba:

- Sin regresión
- Con regresión

¿Qué se prueba?

Pruebo los requerimientos. Pueden ser RNF y RF, debería probar ambos. El problema para la gente de testing es que los RF pueden probarse, los otros no. Si soy una empresa que hago software para otros, no tengo la posibilidad de crear un ambiente de prueba que tenga la configuración del ambiente de producción (donde se llevará a cabo el sistema). ¿Tiene sentido que yo haga pruebas en el ambiente mío siendo que no será llevado a cabo ahí? La respuesta es no. Esta justificación la usan las empresas para no tener un ambiente de pre-producción.

La mayoría de los RNF se dejan para probar en el ambiente de producción.

Aparece otra clasificación que tiene que ver con los tipos de prueba, lo que hacen es descubrir cual es el foco del producto que vos quieres probar con ese. Algunos tipos están enfocados en probar RF y otros en probar RNF.

Para probar los Requerimientos Funcionales tenemos:

- Req en Operación Normal
 - o Negativa
 - o Pruebas de proceso de negocio
 - o Documentación: → Manual de usuario
→ Manual de configuración: para que el software pueda ser utilizado

Para probar los Requerimientos No Funcionales tenemos:

- Prueba de performance
- Prueba de interfaces
- Prueba de escala completa
- Prueba de stress
- Prueba de seguridad
- Prueba de accesibilidad: no solo tiene que ver con la discapacidad de la gente.

todas se pueden probar
a nivel PDU y sistema

CASO DE PRUEBA

Es una situación escrita y detallada que apunta a ser verificación y validación de una determinada funcionalidad. Tiene:

- Escenario a probar (definición paso a paso de lo que debo hacer)
- Precondiciones o condiciones de seteo: debo tener definido que debo cargar en la BD
- Datos de prueba
- Resultado esperado: es lo más importante

Las pruebas deben ser detalladas, no pueden ser pruebas que no sean para un caso específico, con datos concretos.

Llegados al punto de tenemos que probar ¿Cuándo dejar de probar?

Estrategias de prueba:

- Puedo dejar de probar cuando no encuentre más errores, pero siempre va a haber algún error nuevo
- Estrategia de
- Estrategia te doy la cantidad de lo que quedó, prueba lo que puedes

- Estrategia de métricas concretas, hay una cantidad aceptable de severidades de defectos, por ejemplo, ninguno que sea bloqueante, ninguno grave y tantos leves.
- Dejo de probar cuando corré el conjunto de Casos de Prueba

Es un problema cuando dejar de probar, debo dejarlo especificado en el Plan de Prueba. Siempre debemos buscar fortalecer la calidad del producto

COSAS IMPORTANTES!!!!!!

“EL propósito del testing es asegurar la calidad del producto”

Testing NO esta encargada de asegurar de la calidad del producto

Lo único que hace testing es identificar errores y mejorarlos

Testing NO le pone calidad al producto, testing va a hacer evidente que el producto no tiene calidad, ya que no puede agregarle calidad que no tiene. O hiciste las cosas bien o no las hiciste bien, testing te pone en evidencia.

Testing hace CONTROL de calidad, no aseguramiento

A handwritten signature in black ink, consisting of a stylized 'C' followed by a horizontal line and a small upward stroke at the end.